

Évaluation du fine-tuning pour le transfert d'apprentissage en régression

Protocole expérimental

16 juillet 2026

Objectif

L'objectif de cette expérience est d'évaluer le **fine-tuning** comme approche de transfert d'apprentissage dans un problème de régression à partir de données simulées.

Cette expérience se déroule en deux étapes. La première consiste à simuler deux jeux de données, chacun étant généré par un modèle différent. Dans chacun de ces jeux de données, les observations sont réparties entre un **domaine source** et un **domaine cible**. La seconde étape consiste à mettre en œuvre plusieurs stratégies d'apprentissage sur le domaine cible, puis à comparer leurs performances.

Simulation des données

Variables explicatives

On considère quatre variables explicatives : deux variables catégorielles et deux variables continues.

$$X_1 \in \{a_1, b_1\}, \quad X_2 \in \{a_2, b_2, c_2\}, \quad X_3 \in \mathbb{R}, \quad X_4 \in \mathbb{R}.$$

La variable X_1 distingue les deux domaines : une modalité correspond au domaine source et l'autre au domaine cible. La variable X_2 possède trois modalités. Les variables X_3 et X_4 sont continues.

Pour les équations ci-dessous, $i \in \{1, 2\}$ désigne la modalité de X_1 , $j \in \{1, 2, 3\}$ celle de X_2 et k indexe les observations appartenant à la combinaison (i, j) .

Modèles générateurs

Deux mécanismes de génération sont envisagés. Ils permettent de faire varier la complexité de la relation entre les variables explicatives et la réponse.

Modèle 1 : effets principaux et pente propre au domaine

$$\mathcal{M}_1 : \quad Y_{ijk} = \mu + \mu_{1,i} + \mu_{2,j} + (\beta_3 + \gamma_{1,i})X_{3,ijk} + \varepsilon_{ijk}.$$

Ce premier modèle comprend les effets principaux de X_1 et X_2 , ainsi qu'une interaction entre X_1 et X_3 . La pente associée à X_3 peut donc différer entre les domaines source et cible.

Modèle 2 : interactions d'ordre supérieur

$$\mathcal{M}_2 : \quad Y_{ijk} = \mu + \mu_{1,i} + \mu_{12,ij} + (\beta_3 + \gamma_{12,ij})X_{3,ijk} + (\beta_{34} + \gamma_{2,j})X_{3,ijk}X_{4,ijk} + \varepsilon_{ijk}.$$

Le second modèle introduit un effet propre à chaque combinaison des modalités de X_1 et X_2 , une pente de X_3 propre à chacune de ces combinaisons, ainsi qu'une interaction entre X_2 , X_3 et X_4 . Il représente ainsi un mécanisme de génération plus complexe.

Définition des paramètres

- μ est l'ordonnée à l'origine globale ;
- $\mu_{1,i}$ est l'effet associé à la modalité i de X_1 ;
- $\mu_{2,j}$ est l'effet associé à la modalité j de X_2 ;
- $\mu_{12,ij}$ est l'effet associé à la combinaison des modalités i de X_1 et j de X_2 ;
- β_3 est la pente moyenne associée à X_3 ;
- $\gamma_{1,i}$ décrit la modification de la pente de X_3 selon X_1 ;
- $\gamma_{12,ij}$ décrit la modification de la pente de X_3 selon la combinaison (X_1, X_2) ;
- β_{34} est le coefficient moyen de l'interaction entre X_3 et X_4 ;
- $\gamma_{2,j}$ décrit la modification de cette interaction selon X_2 ;
- ε_{ijk} est le terme d'erreur, supposé indépendant et identiquement distribué selon

$$\varepsilon_{ijk} \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2).$$

Proposition de valeurs des paramètres

On associe a_1 au domaine source et b_1 au domaine cible. Les valeurs ci-dessous constituent un premier scénario de simulation : elles créent une différence modérée entre les deux domaines, tout en conservant un rapport signal sur bruit suffisant pour faire apparaître la structure des modèles.

Paramètres du modèle 1

On propose :

$$\mu = 2, \quad \beta_3 = 1,5, \quad \sigma = 1,$$

X_1	a_1	b_1	X_2	a_2	b_2	c_2
$\mu_{1,i}$	0	1	$\mu_{2,j}$	0	-0,75	0,75
$\gamma_{1,i}$	0	-0,5				

Dans ce scénario, à valeurs identiques de X_2 et X_3 , l'ordonnée à l'origine du domaine cible est supérieure d'une unité. En raison du décalage de la distribution de X_3 introduit plus loin, la différence marginale entre les réponses moyennes des deux domaines n'est toutefois pas égale à une unité. La pente de X_3 vaut 1,5 dans le domaine source et 1 dans le domaine cible.

Paramètres du modèle 2

On propose :

$$\mu = 2, \quad \beta_3 = 1,2, \quad \beta_{34} = 0,8, \quad \sigma = 1,$$

$$\frac{X_1}{\mu_{1,i}} \left| \begin{array}{cc} a_1 & b_1 \\ 0 & 0,75 \end{array} \right.,$$

$$\begin{array}{c|ccc} \mu_{12,ij} & a_2 & b_2 & c_2 \\ \hline a_1 \text{ (source)} & 0 & -1 & 1 \\ b_1 \text{ (cible)} & 0 & -0,5 & 1,5 \end{array}, \quad \begin{array}{c|ccc} \gamma_{12,ij} & a_2 & b_2 & c_2 \\ \hline a_1 \text{ (source)} & 0 & 0,4 & -0,4 \\ b_1 \text{ (cible)} & -0,3 & 0,7 & -0,8 \end{array},$$

$$\frac{X_2}{\gamma_{2,j}} \left| \begin{array}{ccc} a_2 & b_2 & c_2 \\ 0 & 0,4 & -0,3 \end{array} \right..$$

Ces valeurs induisent des différences entre les domaines qui dépendent de la modalité de X_2 . La pente de X_3 varie selon chaque combinaison de X_1 et X_2 , tandis que l'effet de l'interaction X_3X_4 varie selon X_2 . Le second scénario est ainsi plus complexe que le premier, tout en gardant des coefficients d'amplitude comparable.

Protocole de simulation des données

Pour chacun des deux modèles, la simulation suivra les étapes suivantes :

1. Construire un tableau de $N = 2000$ observations contenant X_1 et X_2 . Les observations seront réparties de manière aussi équilibrée que possible entre les six combinaisons de modalités de (X_1, X_2) , soit 333 ou 334 observations par combinaison.
2. Ajouter les variables continues X_3 et X_4 . Afin d'introduire un décalage modéré des covariables entre les domaines source et cible, leurs lois conditionnelles à X_1 sont définies par :

$$\begin{aligned} X_3 \mid X_1 = a_1 &\sim \mathcal{N}(0, 1), & X_3 \mid X_1 = b_1 &\sim \mathcal{N}(0,5, 1,2^2), \\ X_4 \mid X_1 = a_1 &\sim \mathcal{N}(0, 1), & X_4 \mid X_1 = b_1 &\sim \mathcal{N}(-0,5, 1,2^2). \end{aligned}$$

Conditionnellement à X_1 , les variables X_3 et X_4 sont générées indépendamment l'une de l'autre et indépendamment de X_2 et du terme d'erreur ε . Le déplacement des moyennes et la légère augmentation de la dispersion dans le domaine cible créent un *covariate shift* tout en maintenant un recouvrement important entre les deux domaines. La variable X_4 est conservée dans les deux jeux de données, bien qu'elle n'intervienne que dans le modèle 2.

3. Calculer la réponse Y à partir de l'équation du modèle considéré, en utilisant les paramètres proposés dans la section précédente, puis ajouter le terme d'erreur ε .

Cette procédure produira deux jeux de données simulées, un pour chaque modèle générateur.

Visualisation des données simulées

Pour chacun des deux jeux de données :

- représenter la distribution de la réponse Y en fonction de X_1 , afin de comparer les distributions obtenues dans les domaines source et cible ;

- représenter séparément les distributions de X_3 et de X_4 en fonction de X_1 , afin de visualiser le *covariate shift* introduit entre les domaines. Les graphiques devront notamment faire apparaître le déplacement des moyennes et l’augmentation de la dispersion dans le domaine cible, tout en permettant d’évaluer le recouvrement entre les distributions source et cible.

Expérience de transfert d’apprentissage par fine-tuning

Dans cette partie, l’objectif est de construire un modèle de prédiction performant sur le domaine cible, tout en disposant d’un nombre limité n_{lim} d’observations cibles pour l’entraînement. L’approche évaluée consiste à préentraîner un réseau de neurones sur les observations du domaine source, puis à l’adapter par fine-tuning à l’aide des n_{lim} observations du domaine cible.

Comme points de comparaison, nous utiliserons :

un réseau de neurones entraîné uniquement sur le même nombre limité d’observations cibles ; un modèle combiné, entraîné simultanément sur les observations du domaine source et sur les n_{lim} observations du domaine cible.

Cette expérience sera menée séparément sur chacun des deux jeux de données simulés. Conformément aux conventions introduites précédemment, a_1 désigne le domaine source et b_1 le domaine cible.

Prétraitement des variables

Les variables utilisées en entrée diffèrent selon la stratégie :

- le modèle préentraîné sur la source, le modèle obtenu par fine-tuning et le modèle cible entraîné *from scratch* utilisent uniquement X_2 , X_3 et X_4 ;
- le modèle combiné utilise X_1 , X_2 , X_3 et X_4 , car il est le seul à être entraîné simultanément sur les deux domaines.

Le prétraitement est défini comme suit :

- pour toutes les stratégies, les modalités de X_2 sont converties en indices entiers, avec $a_2 \mapsto 0$, $b_2 \mapsto 1$ et $c_2 \mapsto 2$, puis transmises à une couche d’embedding ;
- pour le seul modèle combiné, les modalités de X_1 sont converties en indices entiers, avec $a_1 \mapsto 0$ et $b_1 \mapsto 1$, puis transmises à une couche d’embedding spécifique ;
- pour toutes les stratégies, les variables continues X_3 et X_4 sont centrées et réduites à partir des moyennes et écarts-types calculés **uniquement sur le jeu d’entraînement source** :

$$X_j^\star = \frac{X_j - \hat{\mu}_{j,\text{source}}}{\hat{\sigma}_{j,\text{source}}}, \quad j \in \{3, 4\}.$$

Une fois estimées, ces statistiques source sont conservées sans modification et appliquées aux jeux source et cible, d’entraînement, de validation et de test, ainsi qu’aux trois stratégies. Aucune moyenne ni aucun écart-type n’est donc recalculé à partir des données cibles ou du jeu de test. Cette transformation ne supprime pas le *covariate shift*, puisque toutes les observations sont ramenées à la même référence source. La réponse Y n’est pas standardisée.

Etapes de l’expérience

Pour chaque jeu de données, réaliser dix répétitions utilisant des graines aléatoires distinctes. Pour chaque répétition $r \in \{1, \dots, 10\}$:

1. Répartir aléatoirement, en conservant une distribution aussi équilibrée que possible de X_2 , les 1 000 observations du domaine cible ($X_1 = b_1$) entre un jeu de test cible de 500 observations et un réservoir d'apprentissage cible de 500 observations.
2. Répartir de la même manière les 1 000 observations du domaine source ($X_1 = a_1$) entre un jeu d'entraînement source de 800 observations et un jeu de validation source de 200 observations.
3. Calculer les moyennes et écarts-types de X_3 et X_4 uniquement sur le jeu d'entraînement source de la répétition, puis appliquer cette standardisation à tous les sous-ensembles de la répétition.
4. Entraîner, depuis une nouvelle initialisation aléatoire, un MLP sur le jeu d'entraînement source, en utilisant le jeu de validation source pour l'arrêt anticipé.
5. Utiliser les 500 observations du réservoir d'apprentissage cible pour construire aléatoirement des échantillons emboîtés, en conservant une répartition aussi équilibrée que possible de X_2 :

$$S_{10} \subset S_{50} \subset S_{100} \subset S_{200} \subset S_{500}.$$

6. Pour chaque taille $n \in \{10, 50, 100, 200, 500\}$, réserver aléatoirement 20 % de S_n pour la validation cible. Les effectifs d'entraînement et de validation sont respectivement (8, 2), (40, 10), (80, 20), (160, 40) et (400, 100). Les trois stratégies utilisent exactement les mêmes sous-ensembles cibles au sein de la répétition.
7. Affiner une copie du modèle préentraîné (*fine-tuning*) à partir du sous-ensemble d'entraînement cible, sans utiliser X_1 comme entrée, avec un arrêt anticipé fondé sur la validation cible.
8. Entraîner depuis une nouvelle initialisation aléatoire (*from scratch*) un deuxième MLP utilisant uniquement X_2 , X_3 et X_4 sur les sous-ensembles d'entraînement et de validation cibles.
9. Entraîner depuis une nouvelle initialisation aléatoire un troisième MLP utilisant X_1 , X_2 , X_3 et X_4 sur l'ensemble combinant les 1 000 observations source et le sous-ensemble d'entraînement cible. Le sous-ensemble de validation cible guide l'arrêt anticipé. Ce réseau est appelé **modèle combiné**.
10. Évaluer les trois modèles sur le jeu de test cible de la répétition et calculer leur racine de l'erreur quadratique moyenne (RMSE) :

$$\text{RMSE} = \sqrt{\frac{1}{N_{\text{test}}} \sum_{q=1}^{N_{\text{test}}} (Y_q - \hat{Y}_q)^2}.$$

11. Appliquer également la fonction génératrice exacte f aux variables explicatives de ce même jeu de test, sans ajouter un nouveau bruit, et calculer

$$\text{RMSE}_{\text{oracle}} = \sqrt{\frac{1}{N_{\text{test}}} \sum_{q=1}^{N_{\text{test}}} (Y_q - f(X_q))^2}.$$

Visualisation des résultats

Pour chaque modèle générateur, chaque stratégie et chaque valeur de n , représenter par un boxplot les dix RMSE obtenues. Les observations individuelles seront superposées aux boxplots afin de rendre visibles les dix répétitions. Les trois stratégies comparées sont le fine-tuning, l’entraînement cible *from scratch* et l’apprentissage combiné source-cible. Une ligne horizontale noire pointillée indique, sur chaque panneau, la moyenne des dix $\text{RMSE}_{\text{oracle}}$ obtenues sur les dix jeux de test renouvelés du modèle générateur correspondant.

Plancher de RMSE lié au bruit

Comme le terme d’erreur suit

$$\varepsilon \sim \mathcal{N}(0, \sigma^2)$$

et que la réponse Y n’est pas standardisée, la RMSE irréductible dans la population est directement exprimée dans l’unité de Y et vaut

$$\text{RMSE}_{\text{irréductible}} = \sqrt{\mathbb{E}(\varepsilon^2)} = \sigma.$$

Dans le scénario retenu, $\sigma = 1$: le plancher théorique de RMSE vaut donc 1. Pour un prédicteur $\hat{f}(X)$ de la fonction génératrice $f(X)$, l’erreur peut être décomposée, en population, sous la forme

$$\mathbb{E}[(Y - \hat{f}(X))^2] = \sigma^2 + \mathbb{E}[(f(X) - \hat{f}(X))^2],$$

sous l’hypothèse que le bruit est centré et indépendant des variables explicatives. Le premier terme correspond au bruit irréductible ; le second regroupe les erreurs d’approximation et d’estimation du modèle.

Sur un jeu de test fini, même le modèle oracle, qui connaît exactement f , ne présente pas nécessairement une RMSE égale à 1, car la variance empirique du bruit fluctue d’un échantillon à l’autre. Pour la graine de simulation retenue, une RMSE oracle est calculée séparément sur le jeu de test de chaque répétition. Ses fluctuations reflètent les différentes observations incluses dans les dix jeux de test. La moyenne de ces dix RMSE oracle est reportée avec les résultats.

Ces valeurs constituent les références empiriques les plus pertinentes pour interpréter les résultats des MLP. Lorsque le nombre d’observations cibles augmente, un MLP suffisamment flexible et correctement entraîné devrait s’en rapprocher. Il peut néanmoins rester au-dessus à cause de la taille finie des échantillons d’entraînement et de validation, de l’optimisation, de l’arrêt anticipé, du *dropout*, de la régularisation et de sa capacité d’approximation. En outre, augmenter seulement le nombre d’observations sources ne garantit pas d’atteindre ce plancher dans le domaine cible lorsque les relations diffèrent entre les deux domaines.

Arrêt anticipé

Pour chaque entraînement, la fonction de perte est évaluée sur le jeu de validation à la fin de chaque époque. L’entraînement est interrompu si cette perte ne s’améliore pas pendant 20 époques consécutives (*patience* de 20), avec un maximum fixé à 500 époques. Les poids correspondant à la meilleure perte de validation sont ensuite restaurés avant l’évaluation sur le jeu de test.

Le jeu de validation cible est strictement identique pour les trois stratégies au sein d’une répétition. Avec seulement deux observations de validation lorsque $n = 10$, le critère d’arrêt est nécessairement instable ; les répétitions permettront de mesurer la variabilité qui en résulte.

Sauvegarde et visualisation des courbes d'apprentissage

Pour chaque entraînement, la MSE sur le jeu d'entraînement et la MSE sur le jeu de validation sont enregistrées à la fin de chaque époque. Le fichier de résultats associé aux courbes d'apprentissage contient au minimum : le modèle générateur, le numéro de répétition, le type d'entraînement, la taille n de l'échantillon cible lorsqu'elle s'applique, le numéro de l'époque, la MSE d'entraînement, la MSE de validation et l'époque retenue par l'arrêt anticipé.

Avec cinq tailles $n \in \{10, 50, 100, 200, 500\}$, le nombre de courbes est, pour chaque jeu de données simulé :

- $10 \times 2 = 20$ courbes pour le préentraînement source : une courbe d'entraînement et une courbe de validation pour chacune des dix répétitions ;
- $10 \times 5 \times 3 \times 2 = 300$ courbes pour les trois stratégies cibles : dix répétitions, cinq valeurs de n , trois stratégies et deux courbes par entraînement.

Cela représente 320 courbes par modèle générateur, soit 640 courbes pour l'ensemble de l'expérience. Afin de conserver une visualisation lisible, les courbes sont organisées de la manière suivante :

1. **Préentraînement source.** Pour chaque modèle générateur, une figure est divisée en dix panneaux, un par répétition. Chaque panneau contient la MSE d'entraînement et la MSE de validation en fonction de l'époque. L'époque correspondant à la meilleure perte de validation est indiquée par une ligne verticale.
2. **Adaptation au domaine cible.** Pour chaque modèle générateur, une grille synthétique comporte trois lignes, correspondant au fine-tuning, à l'entraînement *from scratch* et au modèle combiné, et cinq colonnes, correspondant aux cinq valeurs de n . Dans chaque panneau, les courbes des dix répétitions sont superposées avec une forte transparence : une couleur est utilisée pour l'entraînement et une autre pour la validation. La médiane des pertes disponibles à chaque époque est ajoutée sous la forme d'une courbe plus épaisse.

L'axe horizontal représente le nombre d'époques et l'axe vertical la MSE. Une échelle logarithmique pourra être utilisée pour l'axe des ordonnées si les pertes couvrent plusieurs ordres de grandeur. Les courbes individuelles sont conservées dans les fichiers de résultats, même lorsque seule la figure synthétique est présentée, afin de pouvoir examiner séparément une répétition présentant un comportement atypique.

Architecture du MLP et paramètres d'entraînement

Une architecture compacte est retenue afin de limiter le surapprentissage, en particulier lorsque peu d'observations cibles sont disponibles. L'embedding de X_2 a une dimension de 2 et est utilisé par toutes les stratégies. L'embedding de X_1 a une dimension de 1 et n'est présent que dans le modèle combiné.

Après concaténation, la partie dense reçoit quatre valeurs pour le préentraînement source, le fine-tuning et l'entraînement cible *from scratch* ($2 + 1 + 1 = 4$), contre cinq pour le modèle combiné ($1 + 2 + 1 + 1 = 5$).

Élément	Valeur proposée
Embedding de X_2	3 modalités $\rightarrow$ dimension 2, pour toutes les stratégies
Embedding de X_1	2 modalités $\rightarrow$ dimension 1, uniquement pour le modèle combiné
Architecture dense — source, fine-tuning et <i>from scratch</i>	4 entrées $\rightarrow$ 32 neurones $\rightarrow$ 16 neurones $\rightarrow$ 1 sortie

Élément	Valeur proposée
Architecture dense — modèle combiné	5 entrées $\rightarrow$ 32 neurones $\rightarrow$ 16 neurones $\rightarrow$ 1 sortie
Activation des couches cachées	ReLU
Activation de sortie	aucune (sortie linéaire)
Fonction de perte	erreur quadratique moyenne (MSE)
Optimiseur	Adam
Taux d'apprentissage — préentraînement, <i>from scratch</i> et modèle combiné	10^{-3}
Taux d'apprentissage — fine-tuning	10^{-4}
Taille des lots	$\min(32, N_{\text{train}})$
<i>Weight decay</i> d'Adam	10^{-4} , appliqué à tous les paramètres entraînaibles (poids, biais et embeddings)
Dropout	0,1 après chaque couche cachée
Nombre maximal d'époques	500
Arrêt anticipé	patience de 20 époques, seuil minimal d'amélioration de 10^{-4}
Initialisation	initialisation de He pour les couches denses ; petite initialisation aléatoire pour l'embedding de X_2 ; initialisation à zéro pour l'embedding de X_1 du modèle combiné

Le taux d'apprentissage plus faible lors du fine-tuning vise à adapter progressivement les poids préentraînés sans dégrader brutalement les représentations apprises sur le domaine source. L'embedding de X_2 et toutes les couches denses restent entraînaibles pendant le fine-tuning.

Rôle de la variable de domaine X_1

La variable X_1 n'est fournie ni au modèle source, ni au modèle fine-tuné, ni au modèle cible entraîné *from scratch*. Dans chacune de ces phases, une seule modalité de X_1 serait observée ; elle n'apporterait donc aucune information permettant de distinguer les domaines.

Le modèle combiné est le seul à observer simultanément les domaines source et cible. Il reçoit donc X_1 en entrée et peut apprendre, au moyen de son embedding, un décalage ou des relations propres au domaine. Cette différence d'entrée est intentionnelle et fait partie de la définition des trois stratégies comparées.

Environnement de mise en œuvre

Le protocole sera implémenté en **Python**. La bibliothèque **PyTorch** sera utilisée pour définir les embeddings et les MLP, entraîner les modèles, appliquer l'arrêt anticipé et produire les prédictions. Les bibliothèques NumPy et pandas pourront être utilisées pour la manipulation des données, tandis que Matplotlib et Seaborn serviront à réaliser les visualisations.

Afin de garantir la reproductibilité, les versions de Python et des bibliothèques seront enregistrées. Les graines aléatoires de Python, NumPy et PyTorch seront fixées et sauvegardées pour les découpages initiaux ainsi que pour chacune des dix répétitions. Si l'expérience est exécutée sur GPU, les options déterministes de PyTorch seront activées dans la mesure du possible.